Strict Architectural Isolation (Task 1):

To successfully execute Step 1: Strict Architectural Isolation (The Control), a scientifically rigorous foundation is established where the only independent variable between the Small Language Model (SLM) and Large Language Model (LLM) is the parameter count of the textual reasoning engine. If the vision mechanisms differ, any downstream performance comparison is scientifically invalid.

The step-by-step execution protocol for standardizing the model architecture to eliminate bias is as follows:

Phase 1.1: Standardizing the Core Backbones (The Qwen Family) To eliminate architectural bias, models are selected from the exact same lineage. Utilizing the Qwen2.5 family ensures that both the SLM (e.g., Qwen2.5-1.5B or 3B) and the LLM baseline (e.g., Qwen2.5-VL-7B or 14B) share the underlying transformer configurations, tokenizer, and vocabulary structures. This is a proven methodology in recent scaling law research, which successfully utilizes the Qwen family (ranging from 0.5B to 14B) to accurately model performance scaling strictly as a function of language model size.

Phase 1.2: Freezing a Unified Vision Encoder To ensure both models perceive the exact same visual stimuli, they are equipped with an identical vision encoder, preventing its weights from changing.

- **The Component:** A robust, pre-trained vision transformer, such as CLIP ViT-L/14, is utilized to extract the spatial features and visual patches from video frames.
- **The Constraint:** The weights of this vision encoder are explicitly frozen across comparative experiments. Recent targeted studies demonstrate that fully fine-tuning the visual backbone during training can significantly degrade performance on tasks requiring fine-grained spatial reasoning, introducing unwanted training variables into the evaluation. By keeping it frozen, it guarantees that the base visual representations passed to the 1.5B model and the 14B model are mathematically identical.

Phase 1.3: Aligning the Vision-Language Projector The connector module is responsible for mapping the visual features from the frozen vision encoder into the text embedding space of the language models.

- **The Component:** Complex, query-based routing mechanisms (like Q-Formers) that might introduce hidden computational biases or varied parameter burdens are avoided. Instead, a simple, two-layer Multi-Layer Perceptron (MLP) projector is initialized.
- **The Execution:** Both the SLM and LLM utilize this exact same MLP architecture to bridge the modalities. This ensures that the translation of visual data into language embeddings is completely uniform across both model scales, leveling the sensory playing field before textual reasoning begins.

Phase 1.4: Isolating the Parameter Scale (N) By locking the vision extraction and projection phases, the total inference compute mathematically isolates the parameter count of the language model (N).

- **The Verification:** The LLM scale is successfully isolated. Any performance deltas observed during downstream task benchmarks can be definitively attributed to the difference in parametric reasoning capacity between the 1.5B/3B engine and the 7B/14B engine.

This perfectly isolated control group serves as the required scientific baseline before introducing a custom data-ingestion pipeline and extreme token compression in the subsequent steps.

Pipeline (Task 2, 3, 4):

To engineer an ultimate, architecture-agnostic data-ingestion pipeline, the most advanced data-centric methodologies from recent literature are combined into a unified workflow. This pipeline strictly decouples visual perception from textual reasoning, guaranteeing that both Small Language Models (SLMs) and Large Language Models (LLMs) are fed a mathematically pristine, identical array of visual information.

The step-by-step execution of this data-ingestion pipeline is as follows:

Step 2: Unified Data-Ingestion Pipeline Engineering (The Foundation) To eliminate black-box preprocessing biases, the pipeline enforces a strict, custom mathematical conversion of raw video into an explicit text-and-vision sequence.

- **Deterministic Frame Extraction:** All default model wrappers are bypassed. Optimized computer vision libraries, such as FFmpeg, are utilized to uniformly extract frames at a strict, standardized rate (e.g., 1 frame per second). This ensures that every model being evaluated receives the exact same starting array of physical pixels, leveling the sensory playing field before reasoning begins.
- **Universal Interleaved Formatting:** All input data is structured into a generalized image-text interleaved template. This mathematically unifies single-image, multi-image, and video formats into one continuous stream, allowing models to process sequential data natively without requiring architectural branching.
- **Spatial-Temporal Demarcation Tokens:** To ensure that the language model explicitly understands the structure of the data without relying on black-box visual projectors, specialized demarcation tokens are injected into the flattened token stream. The entire video sequence is enclosed with `<vid>` and `</vid>` tokens, a `<t>` token is inserted between consecutive frames to explicitly signal temporal dependency, and `\n` is used to organize partitioned high-resolution image grids. This provides a strict, architecture-independent structure to the flattened array.

Step 3: High-Fidelity Data Curation and Semantic Filtering (The Brain)

Instead of relying on noisy, uncurated web-scraped data, an extreme filtering and enrichment phase is enforced within the pipeline to ensure maximum information density.

- **Dual-Consideration Density Filtering:** The information density of the raw dataset is computed by evaluating both intra-cluster concentration (how similar videos are to each other) and inter-cluster separation (how distinct topics are). Videos that fall into high-density, redundant clusters are discarded to mitigate the "sub-scaling law," ensuring every sample provides maximum unique information.
- **Semantic-Based Deduplication:** The global semantics of video captions are extracted using a language model's CLS token (e.g., BERT). A new video is only allowed into the training/testing pool if its maximum cosine similarity to the existing pool falls below a strict threshold.
- **Semi-Automatic Annotation via Teacher Models:** Off-the-shelf dense Vision-Language Models (VLMs) like BLIP-2 are used to generate frame-level captions, and an advanced LLM (like GPT-4V) is then employed to synthesize these into a singular, coherent video-level description. During this process, the LLM is explicitly instructed to discard inconsistent visual noise across frames.
- **VLM Relevance Filtering:** A VLM is deployed as an automatic judge to issue binary alignment scores (true or false) to image-question-answer triplets, aggressively discarding any sample where the visual data fails to perfectly ground the cultural or contextual elements of the text.
- **Synthetic Long-Context Splicing (VIDEOWEAVE):** To train models for long-form reasoning without massive annotation costs, multiple short, independently captioned video clips are synthetically spliced together into a single continuous stream. This forces the model to learn complex temporal tracking and scene-boundary recognition under a fixed compute budget.
- **Discrete Temporal Tokenization:** Continuous floating-point timestamps—which LLMs struggle to tokenize efficiently—are completely eliminated. The video timeline is divided into M equal chunks and mapped to a discrete artificial vocabulary (e.g., `<0>` to `<M>`). This mathematically normalizes time, placing timestamps and text into a unified semantic embedding space to eliminate timestamp quantization errors.

Step 4: Dynamic Visual Compression and Frame Selection (The Filter)

Standard uniform sampling introduces severe perception bottlenecks. This phase utilizes a five-tiered compression strategy to aggressively prune spatial noise and isolate salient temporal events.

- **Tier 1: Query-Agnostic Temporal Segmentation (C-DPCKNN):** First, the unstructured video is parsed into logical "events." Frame-level representations are extracted using a CLIP vision encoder, local density and distance indices are calculated, and event

boundaries are explicitly set at minimum density peaks. This groups consecutive frames without blurring boundaries or interleaving disparate scenes.

- **Tier 2: Differential Keyframe Selection (DKS):** To identify the most critical frames, the cosine similarity between each frame's visual embedding and the textual query's embedding is calculated. A greedy algorithm is used to sort frames by their query-relevance score, selecting the top frames while enforcing a strict temporal distance threshold (τ) to guarantee that selected keyframes are both highly relevant and temporally distinct, avoiding redundant clusters.
- **Tier 3: Temporal Dynamic Compression (Supplementary Frame Selection):** To complement the keyframes, a small ratio of base frames is sampled uniformly ($\alpha=0.85$), and then supplementary frames are iteratively selected from the unchosen candidates that possess the lowest cosine similarity (highest semantic variation) to the existing set. This captures crucial, rapid actions that rigid selection algorithms might skip.
- **Tier 4: Adaptive Spatial Compression (Pruning & Merging):** For each selected frame, uninformative spatial background noise is eliminated using a two-step method:
    - **Magnitude-Based Pruning:** The L_2 norm (activation magnitude) of all visual patch tokens in a frame is computed. This is used as a proxy for visual salience, and tokens that fall below a predefined retention ratio ($\beta=0.625$) are aggressively pruned.
    - **Semantic Token Merging:** The remaining high-saliency tokens are grouped using a cosine similarity threshold ($\tau=0.9$). Highly redundant tokens (e.g., large patches of identical texture) are consolidated into a single representative token via mean pooling.
- **Tier 5: Differential Feature Merging (DFM) for Non-Keyframes:** Instead of dropping non-keyframes entirely (which destroys continuous context), they are compressed down to a single representative token. A differentially weighted pooling operation is applied where patches receive higher weights if they are highly relevant to the query and have low temporal redundancy when compared to the preceding keyframe.

The Final Output: By executing this ultimate data-ingestion pipeline, the input is transformed from an unmanageable, noisy video file into a mathematically dense, perfectly sequenced array of discrete temporal tokens and non-redundant spatial features. Both the SLM and the LLM are evaluated purely on their cognitive ability to reason over this pristine data, entirely free from architectural or preprocessing biases.

Inference Compute (FLOP) Standardization (Task 5):

To flawlessly execute Step 5: Inference Compute (FLOP) Standardization, a strict mathematical framework is implemented to govern how tokens and parameters are allocated during the experiment. Standard evaluations often invalidate their findings by allowing massive models to process dense visual tokens, which explodes the required FLOPs and makes edge-device deployment impossible.

To scientifically prove the Core Hypothesis—that a highly curated data pipeline allows a Qwen2.5-1.5B (SLM) to match a Qwen2.5-VL-7B/14B (LLM) at the edge—both models are forced to operate under the exact same computational ceiling.

The step-by-step execution protocol for standardizing inference FLOPs is as follows:

Phase 5.1: Defining the Mathematical FLOP Equation First, the theoretical compute cost is programmatically tracked for every single inference run. The total inference FLOPs for a Vision-Language Model (VLM) is determined by both the language model and the vision encoder.

- **The Standard LLM Equation:** For the language reasoning backbone, inference FLOPs are calculated as $\mathcal{O}(N \times (Q + V))$, where N is the parameter count of the LLM, Q represents the text prompt tokens, and V represents the visual tokens.
- **The Complete Video VLM Equation:** Because video is being processed, the vision encoder's cost—which is often biasedly ignored in other research—must be included. The strict per-example inference compute cost $c(x)$ is defined as $c(x) = 2x_T(x_M x_W + x_N x_V)$. Here, x_T is the number of video frames, x_M represents the vision model parameters, x_W represents the visual features output by the vision model, x_N is the LLM parameters, and x_V is the final visual tokens per frame.
- **Action:** This exact equation is coded into the orchestration script to act as a hard constraint. If an inference request exceeds the predefined $c(x)$ budget, it is actively blocked.

Phase 5.2: Establishing the Anchor Compute Budget The baseline computational ceiling is established using the Small Language Model (SLM) processing a dense, high-fidelity token stream.

- **Action:** The Qwen2.5-1.5B or 3B model is set up. It is allowed to ingest a densely sampled array of video frames (e.g., $x_T = 16$ to 32 frames) and a relatively dense spatial token count per frame (e.g., $x_V = 144$ tokens per frame). The total FLOPs required for this SLM to process the query is calculated. This specific FLOP value becomes the absolute Inference Compute Budget for the rest of the experiment.

Phase 5.3: Applying Extreme Token Compression to the LLM To fairly test the Qwen2.5-VL-14B (or 7B) LLM, it is restricted to the exact same FLOP budget calculated for the 1.5B model. Because the parameter count N (14B) is exponentially larger, the visual tokens V must be aggressively slashed to compensate.

- **The Inference-Optimal Law:** Recent scaling laws demonstrate that for a fixed compute budget on visual reasoning tasks, the optimal strategy is to utilize the largest possible LLM but heavily compress the visual input down to as few as 1 to 4 tokens. The average downstream error increases approximately 5$\times$ faster when reducing LLM parameters than when reducing visual tokens.

- **Action:** An extreme token reduction algorithm (such as 1TPI - One Token Per Image, or a convolutional TokenPacker) is implemented to compress the 14B model's visual input down to 1 to 4 tokens per frame. It is ensured that the calculation $2x_T(x_M x_W + x_{14B} \times x_{1\text{-to-}4}) \approx$ The Anchor Compute Budget.

Implementing Dynamic Scaling and Edge Validation (Tasks 5.4 – 5.6):

Here is the step-by-step execution protocol for the remaining phases, translated into a formal, general tense framework:

Phase 5.4: Implementing Dynamic Scaling for Long-Text Queries (Q) The strict rule of heavily compressing visual tokens for the LLM has one mathematical exception that must be coded into the pipeline: dynamic text lengths.

- **The Phenomenon:** If the user text prompt (Q) is massive (e.g., a long system prompt or dense RAG context), the computational cost incurred by the text tokens already dominates the FLOPs. At this point, increasing the number of visual tokens has a comparatively minor impact on the overall computational burden.
- **Action:** A dynamic threshold is programmed into the pipeline. If the text token length Q exceeds a certain threshold, the pipeline dynamically allows the LLM to process a slightly higher number of visual tokens (e.g., relaxing compression from 4 tokens up to 16 tokens) to maximize compute-optimal performance without breaching the ceiling.

Phase 5.5: Task-Routing Bypass for Detail-Heavy Tasks The inference-optimal law of extreme visual compression works flawlessly for general video reasoning and multi-modal chat. However, aggressive compression destroys performance on tasks requiring high-frequency spatial details.

- **The Failure Mode:** Tasks like Optical Character Recognition (OCR) or dense document understanding exhibit the complete opposite scaling law: they require as many visual tokens as possible because extreme compression destroys the text and structural details.
- **Action:** A lightweight "Task-Router" or intent classifier is integrated at the start of the pipeline. If the user query is classified as "Text Recognition/OCR," the pipeline temporarily suspends the strict FLOP standardization and feeds dense tokens to both the SLM and LLM to prevent catastrophic information collapse.

Phase 5.6: Physical Edge-Hardware Validation Theoretical FLOPs do not perfectly translate to physical wall-clock time due to hardware overhead, memory bandwidth, and the separation of prefill vs. decode stages.

- **Action:** The FLOP-standardized 1.5B model (processing dense tokens) and the 14B model (processing compressed tokens) are deployed onto the physical edge device (e.g., Jetson Orin Nano). The Time-to-First-Token (TTFT)—which measures the compute-bound prefill stage processing the video—and the Peak VRAM Memory—which captures the KV Cache footprint—are logged.

- **The Final Proof:** If the 1.5B SLM processing pristine, data-engineered dense tokens matches the reasoning accuracy of the 14B LLM while staying under the identical FLOP budget and avoiding Out-of-Memory (OOM) edge crashes, the core thesis is empirically proven.

Optimized Orchestration and Edge-Deployment Testing (Task 6):

To flawlessly execute Step 6: Optimized Orchestration and Edge-Deployment Testing, any software or network bottlenecks that could falsely penalize the model's actual cognitive speed must be systematically eliminated, and the dual-stage inference latency must be meticulously tracked on physical edge hardware.

The step-by-step execution protocol for this phase of the framework is as follows:

Phase 6.1: Mitigating Data I/O and Orchestration Bottlenecks If the orchestration pipeline involves moving massive video files between separate modules (e.g., from a client listener to a cloud API), the data transfer latency will entirely overshadow the model's inference time, invalidating edge-viability metrics.

- **Action:** A fully local deployment on the edge device must be executed to prevent network I/O from skewing the results. The video must be processed into image sequences and compressed by the pipeline locally before being fed into the SLM.
- **Standardize the Inference Engine:** To ensure that different software wrappers do not introduce artificial latency bias, both the SLM and the LLM are deployed using a standardized, highly optimized open-source inference engine like `llama.cpp` or `vLLM`.

Phase 6.2: Physical Edge-Hardware Setup The models are deployed onto heterogeneous edge devices to capture a realistic spectrum of hardware constraints.

- **Action:** Dedicated robotic edge platforms with active cooling (e.g., the Jetson Orin NX or Jetson Orin Nano, which typically feature 16GB of DRAM) are utilized alongside consumer smartphones equipped with flagship SoCs (e.g., Snapdragon 8 Gen 1+ or Google Tensor).

Phase 6.3: Tracking the Prefill Stage (Compute-Bound Latency) The prefill stage involves ingesting the text prompt alongside the massive sequence of compressed video tokens.

- **The Metric:** The Time to First Token (TTFT) or Prefill Latency is logged.
- **Hardware Dynamics:** Because the prefill stage processes tokens simultaneously, it is heavily compute-bound and benefits massively from hardware parallelism. For example, research shows that deploying on a GPU-enabled edge device like the Jetson Orin can process the prefill stage up to 40× faster than a CPU-only smartphone.
- **Architectural Bottlenecks:** Within the model itself, the time spent in the Feed-Forward Neural Network (FFN) and Attention layers is tracked, as matrix-vector multiplications (`mul_mat_vec_q`) in these layers can account for over 80% of the total prefill latency.

Tracking the Decode Stage and Stress-Testing (Tasks 6.4 – 6.5):

Here is the step-by-step execution protocol for tracking the decode stage and stress-testing the hardware, translated into a formal, general tense framework:

Phase 6.4: Tracking the Decode Stage (Memory-Bound Latency) Once the video context is understood, the model begins generating its textual answer sequentially, one token at a time.

- **The Metric:** The Per-Token Decode Latency and overall Throughput (Tokens Per Second) are logged.
- **Hardware Dynamics:** The decode stage is sequential and highly memory-bandwidth-bound. Consequently, the massive GPU advantage seen in the prefill stage drops significantly here (e.g., the Jetson GPU may only be 1.8× faster than a smartphone CPU during decoding).
- **The Quantization Advantage:** To optimize decode throughput, quantization (e.g., 4-bit precision) is applied. Because decoding requires constant memory access per token, quantization is vastly more effective in reducing latency during the decode stage (reducing latency by up to 75%) than in the compute-heavy prefill stage. Regular bit-widths (like Q4_K_M) are used to avoid the alignment padding overheads caused by irregular precisions.

Phase 6.5: Stress-Testing Memory Footprints and Thermal Throttling Processing video context inherently triggers severe hardware stress that standard NLP benchmarks ignore. Two critical failure modes are tracked during edge deployment:

- **KV Cache Explosion:** Peak VRAM is monitored closely. For continuous video understanding, the dynamic KV cache and compute buffer will completely dominate memory usage—frequently accounting for up to 85% to 87% of total memory at long context lengths.
- **Thermal Latency Spikes:** Device temperatures and subsequent latency fluctuations are tracked. Passively cooled devices (like smartphones) will likely experience severe thermal throttling and massive latency spikes after generating just 1 to 10 tokens, stabilizing only at heavily degraded speeds. This is compared against actively cooled Jetson platforms, which can maintain consistent token generation for much longer sequences before fluctuating.

The Final Proof: By rigidly tracking these disparities, it is empirically proven whether the data-engineered SLM can survive the KV cache explosion and thermal constraints of real-world video inference that would otherwise crash an LLM baseline.

Holistic Matrix Evaluation (Task 7):

To flawlessly execute Step 7: Holistic Matrix Evaluation, the data-engineered Small Language Model (SLM) and the baseline Large Language Model (LLM) are evaluated across two distinct

axes: their cognitive reasoning capabilities on standardized datasets, and their physical hardware limitations during edge-device deployment.

The step-by-step execution protocol for finalizing the project and definitively proving the Core Hypothesis is as follows:

Phase 7.1: Measuring Cognitive Intelligence (Task Accuracy) To prevent evaluation bias, the models are tested across a spectrum of temporal complexities using both strict Exact-Match and dynamic LLM-as-a-Judge protocols.

Standardized Benchmarking Matrix:

- **Video-MME:** This is used to evaluate comprehensive visual cognition. It assesses video analysis capabilities across 30 distinct sub-fields, requiring models to process long visual contexts without relying solely on subtitle information.
- **MVBench:** This is deployed to evaluate strict temporal reasoning. It targets 20 challenging video understanding tasks (such as state change and action sequencing) that are intractable with single-frame analysis, forcing the model to rely on multi-frame reasoning.
- **LongVideoBench & ActivityNet-QA:** LongVideoBench is used to test extreme long-context retrieval for videos up to one hour in duration, and ActivityNet-QA is utilized specifically for fine-grained action-related queries.

LLM-Judged vs. Exact Match Criteria: For Multiple-Choice Questions (MCQ), standard accuracy metrics are utilized. However, for open-ended video QA and detailed captioning, rigid exact-match metrics fail to capture semantic accuracy. Therefore, an **LLM-as-a-judge framework** is deployed. A highly capable model like GPT-4o-mini is prompted to rate the open-ended outputs, and both the MCQ accuracy and the LLM judge's scores are standardized to a strict (0, 100) scale to facilitate a balanced average across all metrics. If there is a need to avoid closed-source API limits, an open-source evaluator like Vicuna-1.5 (13B) can be utilized, as it has been proven to yield highly consistent evaluation trends in correctness, detail orientation, and temporal understanding.

Measuring Physical Systems-Level Efficiency (Computational Cost) (Phase 7.2):

Theoretical intelligence is insufficient if the model crashes the edge device. Both the Small Language Model (SLM) and the FLOP-matched Large Language Model (LLM) are deployed onto physical edge hardware (e.g., Jetson Orin), and critical hardware metrics are meticulously logged.

- **Time to First Token (TTFT) / Prefill Latency:** The TTFT is logged to measure the model's efficiency during the compute-bound prefill stage. Because video understanding requires ingesting a massive sequence of visual tokens all at once, the prefill stage heavily dominates on-device LLM inference latency. This metric proves whether the data-centric pipeline successfully prunes enough visual noise to prevent prefill bottlenecking.

- **Tokens Per Second (TPS) / Decode Latency:** Once the video is processed, the per-token decode latency is tracked. Unlike the compute-bound prefill stage, the decode stage is sequential and strictly memory-bandwidth-bound. Logging the TPS ensures the SLM can maintain a stable, real-time conversational generation speed without triggering thermal throttling.
- **Peak VRAM & KV Cache Utilization:** The absolute maximum memory footprint during the inference loop is continuously monitored. While static memory is dictated by model parameter size, the dynamic KV cache explodes as video context lengthens, often becoming the primary bottleneck for continuous video understanding (frequently accounting for up to 85% to 87% of total memory at long context lengths). This metric definitively proves whether the larger 7B/14B LLM baseline hits an Out-Of-Memory (OOM) wall on edge hardware, while the optimized 1.5B SLM remains within safe VRAM limits.
- **Sustainability Metrics (Optional but Recommended):** To provide a truly holistic evaluation of edge viability, specialized libraries are utilized to log total Energy Consumption (kWh) and CO$_2$ Emissions per inference request, as edge devices operate under strict battery and thermal constraints.

Establishing the Pareto-Optimal Efficiency Frontier (Phase 7.3):

The final step involves synthesizing the cognitive and physical metrics into a mathematical proof of the core thesis.

- **Plotting the Trade-Offs:** The total inference FLOPs of a Vision-Language Model are governed by the equation $\mathcal{O}(N \times T)$, where N is the language model parameter count and T is the total number of visual and textual tokens processed.
- **The Compute-Optimal Frontier:** The average downstream task error (from Phase 7.1) is plotted against the actual inference compute cost (from Phase 7.2). The empirical results of the 1.5B SLM (processing pristine, dense visual tokens via the custom pipeline) are mapped against the 7B LLM (which is forced to aggressively compress its visual tokens to stay under the exact same FLOP budget).
- **The Final Proof:** By solving for this constrained optimization, the compute-optimal frontier is plotted. If the results show that the 1.5B SLM achieves equivalent or superior average scores on Video-MME and MVBench compared to the FLOP-matched 7B LLM, while simultaneously operating with drastically lower TTFT and Peak VRAM on the physical device, the Core Hypothesis is definitively proven. This demonstrates that data-centric visual ingestion allows a compact SLM to punch far above its parameter weight class, achieving true Pareto-optimal efficiency at the edge.